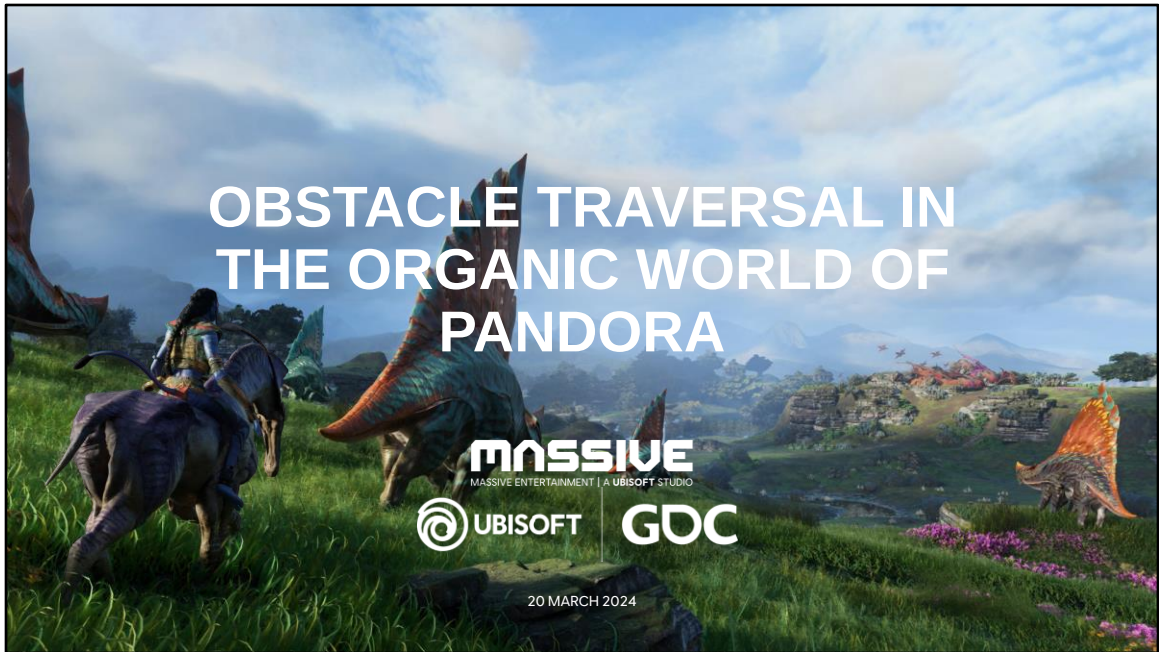
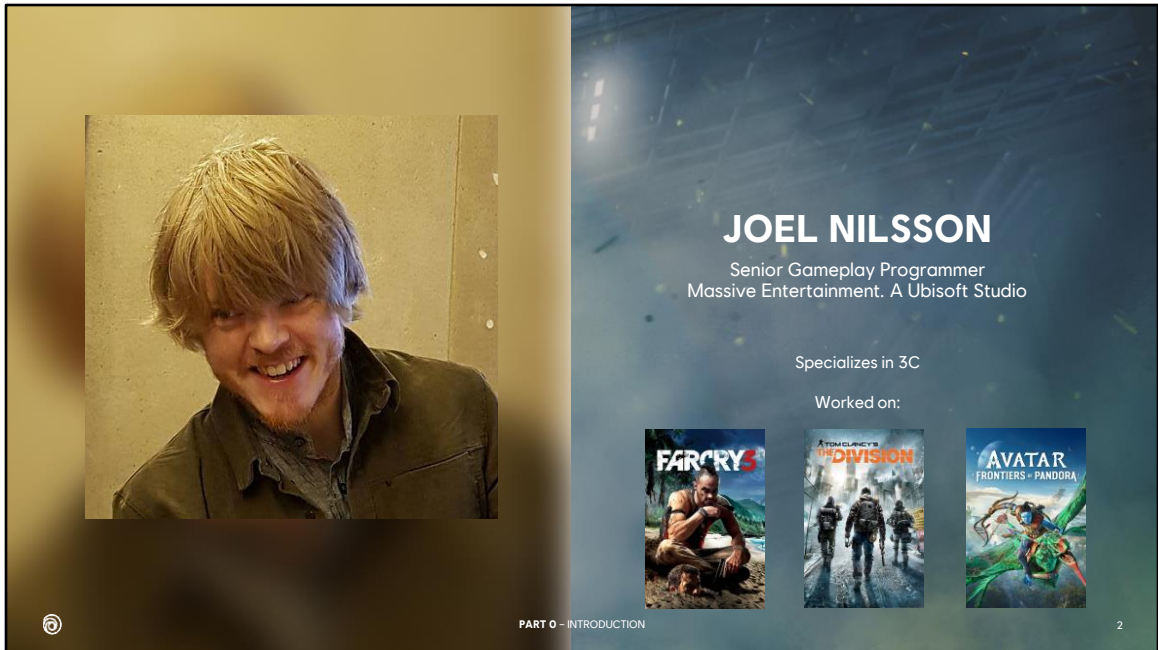




Hello. Welcome to this talk about Obstacle traversal in the organic world of Pandora



JOEL NILSSON
Senior Gameplay Programmer
Massive Entertainment. A Ubisoft Studio

Specializes in 3C

Worked on:

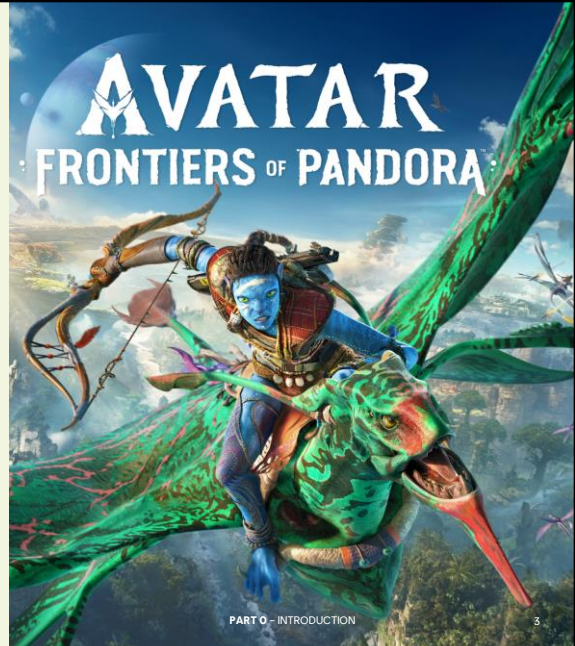
FAR CRY 3 **TOM CLANCY'S THE DIVISION** **AVATAR FRONTIERS OF PANDORA**

 **PART 0 - INTRODUCTION** 2

First a little bit about me. I'm a gameplay programmer at Massive Entertainment, specializing in 3Cs, that's Camera, Character and Controls and I've worked on games like Far Cry 3, The Division and most recently Avatar: Frontiers of Pandora.

THE GAME

- RELEASED DECEMBER 2023
- SET IN JAMES CAMERON'S AVATAR UNIVERSE
- TAKES PLACE ON THE LUSH MOON OF PANDORA
- FIRST PERSON ACTION ADVENTURE WHERE YOU PLAY AS A NA'VI



Which is the game we'll be talking about today. It was released in December of last year. It's set in on James Cameron's Avatar universe and you get to explore the lush moon of Pandora as a Na'vi in a first person action adventure.

3C Pillars

- PANDORA FIRST
- JOY OF MOVEMENT

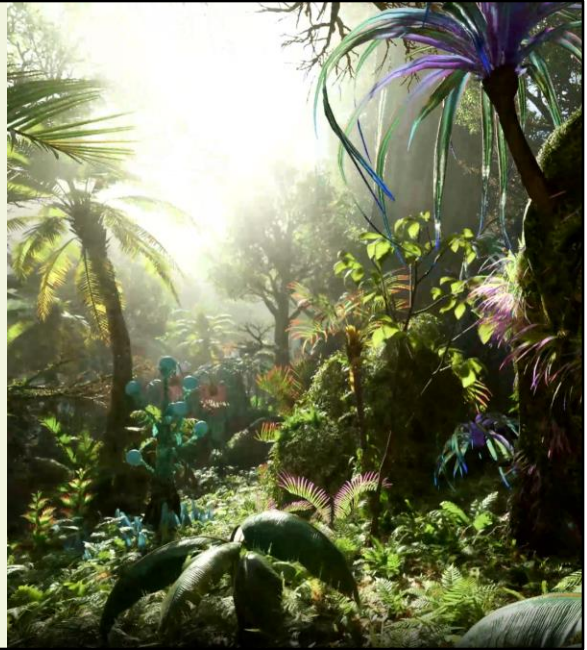


So, when we started making this game we had two main pillars which we built the 3C experience around.

Pandora first and joy of movement

PANDORA FIRST

- THE WORLD HAD TO LOOK BEAUTIFUL AND ORGANIC
- ORGANIC = CHAOTIC - NO STRICT METRICS OR MARKUP
- A MORE GENERIC AND PROCEDURAL APPROACH WAS NEEDED



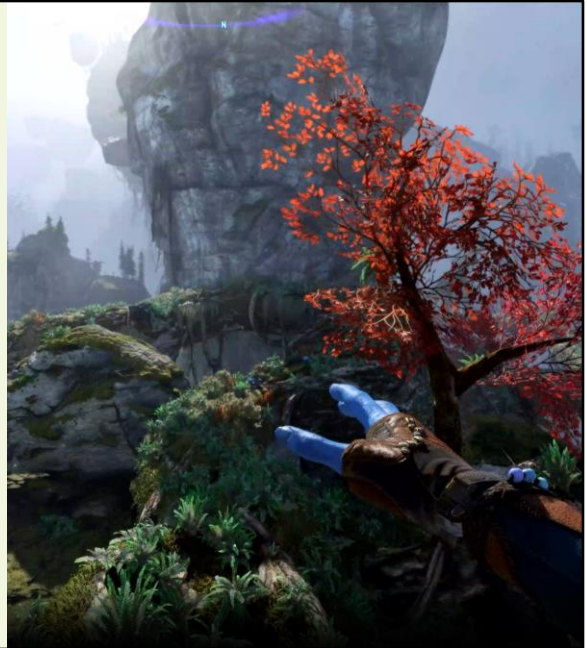
Probably the strongest fantasy of any Avatar game is to visit Pandora, and we wanted to get as close to that as possible, so the world had to look as beautiful and organic and dense as in the movies.

But this came with some caveats. A jungle is essentially a mess. It's made up of randomly placed, organically shaped objects and both of these factors makes it hard to analyze.

So after some testing we realized that the metrics and markup approach wasn't going to work. We needed something that could handle anything that random could throw at us. Which is why we chose to go with the much more general approach I'll talk about today.

JOY OF MOVEMENT

- SENSE OF SUPER-HUMAN POWER AND SPEED
- MOVE LIKE YOU WERE BORN A NA'VI
- SMOOTHNESS AND FLOW PRIORITIZED



If the first pillar was about the world, the second one is about the player. We wanted the player to experience the same kind of joy of movement that Jake does in the first movie when he leaves his wheel chair and finds himself in this amazing alien body.

And the word "Joy" is important here. Joy is not about being challenged to within an inch of your abilities. It is about sense of exciting but confident flow. We wanted you to feel like you were born to do this.

So as much as we like parkour movement, we didn't want the player to have to learn parkour. We wanted them to feel competent from the start, and to grow at their own pace. So we prioritized smoothness and flow over spectacular moves.

OBSTACLE TRAVERSAL IN ACTION



PART 0 - INTRODUCTION

7

Now let's look at our traversal system in action!

WHY THIS TALK?

- Getting stuck is annoying
- Markup is labor intensive and error prone
- Generic obstacle traversal is complex
- Sharing notes helps



So why this talk?

Well, first of all because getting stuck is annoying. And because markup is labor intensive and error prone.

But of course our alternative, generic obstacle traversal, is a hard problem, which is why I think it helps if we share notes. And that's what I want to do today.

WHAT IS THIS TALK ABOUT?

- Finding and getting past obstacles in ground traversal
- One example of how to solve this
- The philosophy behind it
- How it works and why it works
- It will get a bit technical. But don't worry



PART 0 - INTRODUCTION

9

So what I'm going to talk about is a way of finding obstacles and getting past them when traversing the world on foot. It's not the only way. It might not be the best way. But it is one way and I hope you will find it useful to see how it works.

So, I'll explain some of the philosophy behind it, how it works and WHY it works because I think that the better you understand something the more useful it is.

I should also mention that this will get a bit technical. But don't worry if it's not all clear right now. The important thing is the general principles and the slides will be available later.

REALITY CHECK/DISCLAIMER

- What follows are principles
- A bit idealized and simplified
- In real life there are buffers and tweaking
- The real system is 3D. 2D for explaining



Before I move on I just want to point out that what follows will be the basic principles of what we do

It will be a bit idealized and simplified.

In reality of course there is imprecision and buffers and tweaking to make it work. So keep that in mind

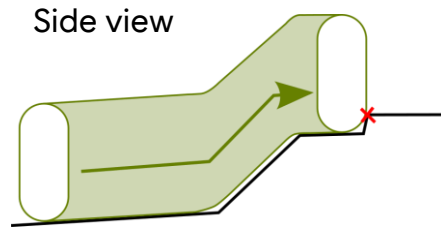
Finally, in reality the system is in 3D of course, but we'll be using 2D for most of the explanation, because I think it's easier to understand that way.



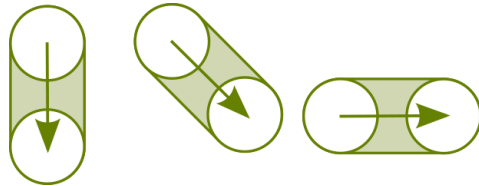
To make it a little easier to answer the question "Why it works" I thought I'd start with some sphere cast basics. This might sound to some of you like I'm just stating the obvious, but bear with me. This will be relevant later.

MOVEMENT AND CAPSULES

- We drag a capsule through the world and are stopped by unwalkable slopes



- A sphere cast is a capsule



PART 1 - SPHERE CAST BASICS

12

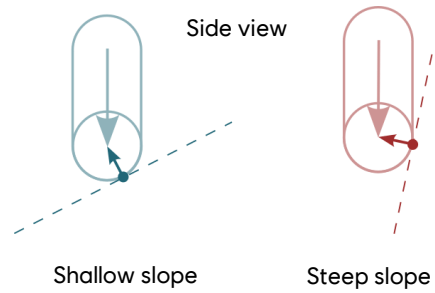
In our game we use a capsule for movement. We just drag it along the surface of the world and if it hits something it can't get past it stops.

We chose to use a capsule rather than a pencil partly because it has natural smoothing but also because a sphere cast becomes a capsule by extrusion.

And sphere casts have some very interesting properties that we can exploit.

CASTS AND NORMALS

- Casts give us a hit point and a normal
- The normal corresponds to a slope
- Shallow slopes we can stand on
- Steep slopes stop us



Now, let's look at what we get when we do a cast. We get a hit point and a normal.

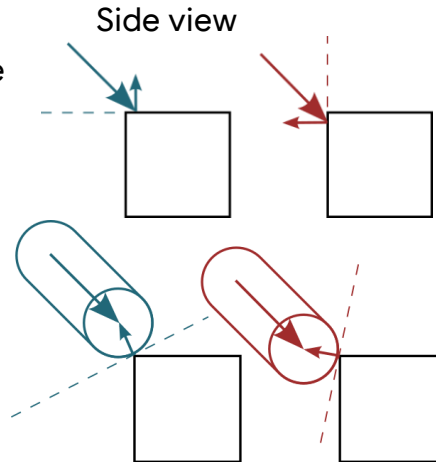
The normal, of course, corresponds to a slope. The slope that it is normal too.

Some normals correspond to shallow slopes that we can stand on and some correspond to steep slopes that we slide off.

CASTS AND NORMALS

- Ray cast normals tell us about the world

- Sphere cast normals tell us about the sphere's relation to the world



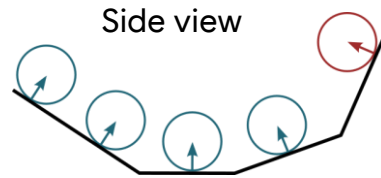
So, now, let's look at an interesting difference between ray casts and sphere casts.

If you cast a ray you will get a hit point and you will get a normal and the normal tells you what the normal of the surface you hit was. The normal says nothing about the ray. It just says something about the world.

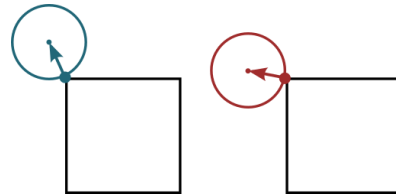
If you cast a sphere you will get a normal that tells you what the normal of the sphere at the hit point is. And if it hits the corner, like here, it doesn't even correspond to a normal in the world. It only tells you the sphere normal.

CASTS AND NORMALS

- Surface Normal = sphere normal



- Sphere hit point -> normal
- Normal -> sphere hit point



Sometimes we do hit a surface of course. And in that case the normal we get is the normal of the surface, but that normal is also always the normal of the sphere at the hit point. So getting the sphere normal is the only constant.

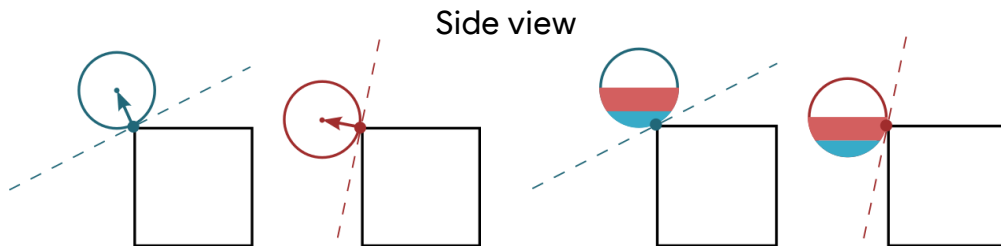
This leads us to another interesting property of sphere casts. If we have the hit point on the sphere we also know what the normal is. We just draw a line from the hit point to the center of the sphere.

But also, if we have the normal, we know where the hit point is. Because only one point on the sphere surface has that normal.

In other words, hit point and normal are interchangeable.

WALKABLE SLOPES

Hit area gives normal and slope properties



PART 1 - SPHERE CAST BASICS

16

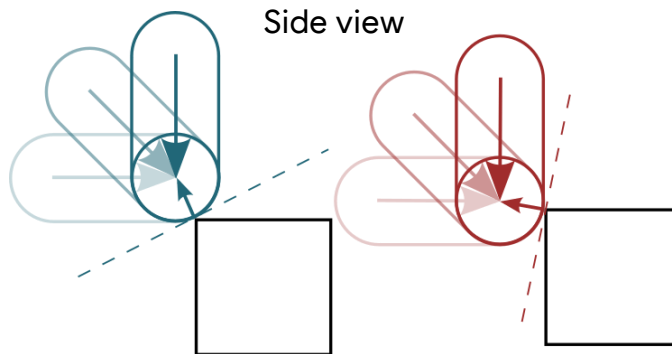
This means that if a certain group of normals have a certain property, we can associate that property with the area of the sphere where these normals occur. Because they're interchangeable.

What we're interested in here is that normals that represent shallow slopes occur at the bottom of the sphere, whereas those that represent steeper slopes occur higher up.

You can see here that the sphere slopes more steeply in the red area than in the blue area.

WALKABLE SLOPES

Cast direction doesn't matter for the normal.



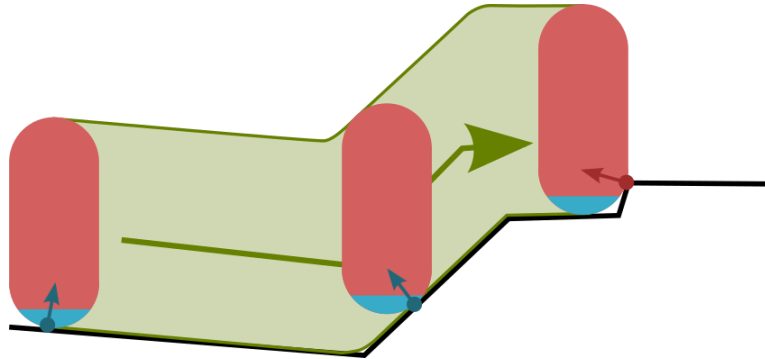
PART 1 - SPHERE CAST BASICS

17

Before we get back to the capsule I want to point out one more thing. The direction from which a sphere hits doesn't matter for the normal, and what it means is that we'll get the same normal whether we drag it along a surface or cast it from above as long as the surface is the same.

WALKABLE SLOPES

The capsule hit point shows path walkability



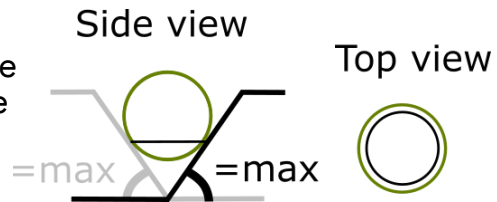
PART 1 - SPHERE CAST BASICS

18

So let's look at this capsule dragging again. As you can see, as long as it can move it hits the bottommost part of the capsule, but when it stops it's because it hits higher up.

WALKABLE SLOPES

- A walkable slope is a slope that is below a certain max angle
- All hits on a slope with exactly the max angle hit in a ring around the sphere
- We call this the max angle ring



This brings us to some central definitions for our system.

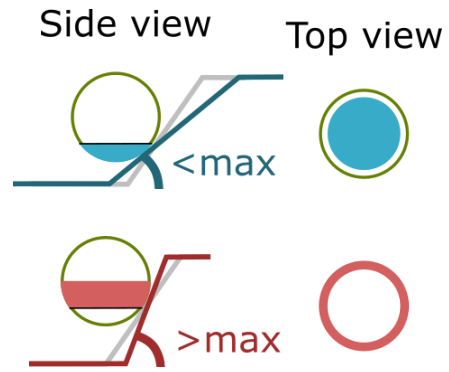
First, a walkable slope is a slope that is below a certain max angle.

All hits on a slope that is exactly the max slope occur on a horizontal ring around the sphere. It might be easier to see if you imagine the slope in this picture rotating around the sphere. Anywhere it touches is the ring.

We're going to call this ring the max angle ring.

WALKABLE SLOPES

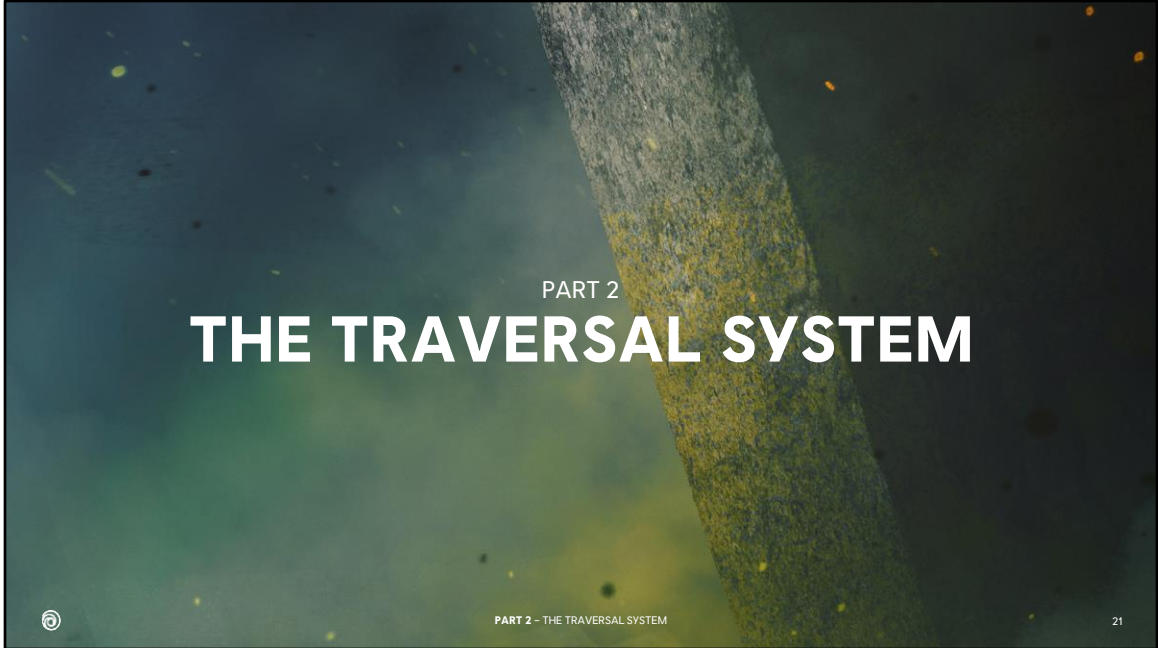
- Any hit below the max angle ring is shallower and therefore walkable
- We call this area the **walkable circle**
- Any hit above the max angle ring is steeper and therefore unwalkable.
- We call this area the **unwalkable ring**



Now, any hit below this max angle ring must be on a shallower slope and therefore on a walkable slope. We'll call this area the walkable circle

Conversely any hit above it must be steeper and therefore unwalkable. We'll call this area the unwalkable ring.

These concepts will be important later.



Ok. Thank you for staying with with me. Now we can move on to the actual traversal system.

PHILOSOPHY

- Complex problems require simple solutions
 - The more complex the problem is, the more it's worth investing in simplifying the solution
- One system for both stepups and mantles
 - Because a unified pipeline is simpler to manage
- Respect for player intentions
 - If the player wants to turn, let them turn. If they want to abort, let them abort.



Let's start with the pilosophy behind it.

A core principle was that complex problems require simple solutions. What we mean by that is simply that the more complex the problem, the more it's worth investing in simplifying the solution. Because we didn't want to debug a complex system trying to solve a complex problem.

This led us to settle for one system to solve all obstacle traversal, both stepups and mantles. Because one system and one decision pipeline is simpler than many that need to cooperate.

Finally another core principle was to always respect player intentions. Because if the player wants to turn, let them turn. If they want to abort, let them abort. Again it's about the player being in control of this amazing body.

PARTS

- Detection
 - When and where to move
- Movement
 - How we perform the movement
- Extensions
 - Simple improvements beyond the core system



Moving on now to the more practical part.

We'll start out by looking at detection, that's when a where to move. This will be the biggest part because it's the trickiest one.

Then we'll look briefly at how we move and how we make it feel nice.

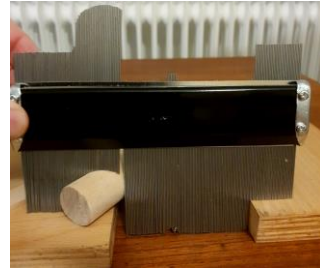
Finally we'll have a look at some extentions we made to improve the system.



But let's start with the biggest bit. Detection

THE PROFILE GAUGE

- Creates a simplified picture of complex geometry
- Allows analysis of a slice without the geometry



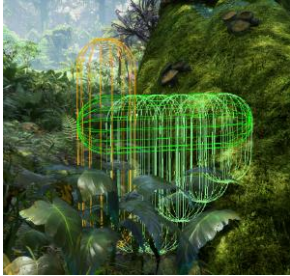
PART 3 - DETECTION

25

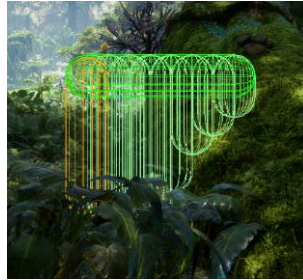
As I said we tried to simplify as much as possible. To do this we took inspiration from this thing: The profile gauge. You might recognize it from carpentry. The great thing about this tool is that it allows you to create a simplified 2D picture of complex 3D geometry.

You can the extract this little slice of the world and analyse that rather than the whole bewildering thing.

OUR PROFILE GAUGE



Walking



Sprinting



Jumping



PART 3 - DETECTION

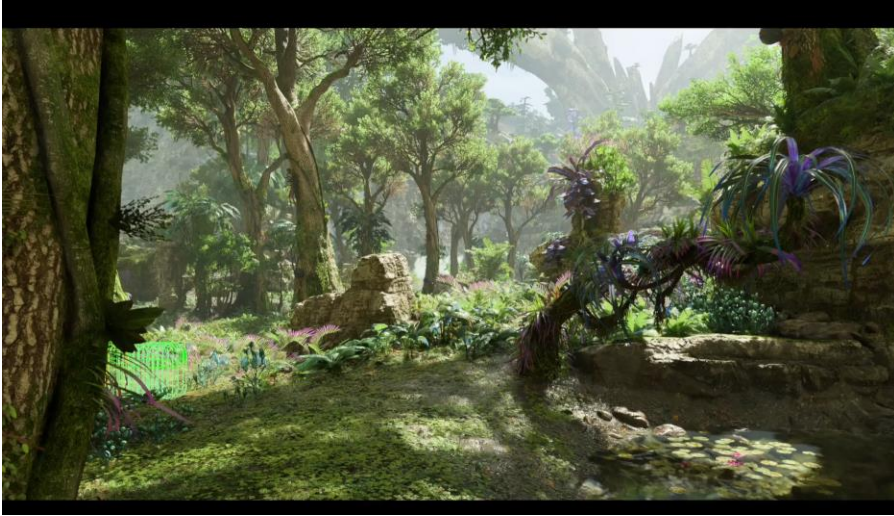
26

So here you have it. Our profile gauge as it looks in the game. The player in orange and the profile gauge casts are in green

As you can see it looks pretty similar to the real thing, except it's got fewer and thicker pins. You can also see here that the faster we move the further and higher we search. And when we jump we search even further and higher so we can do high mantles.

You can also see that the further we search, the more pins our profile gauge gets. Here, from 5 when walking to 7 when jumping fast.

OUR PROFILE GAUGE IN ACTION



PART 3 - DETECTION

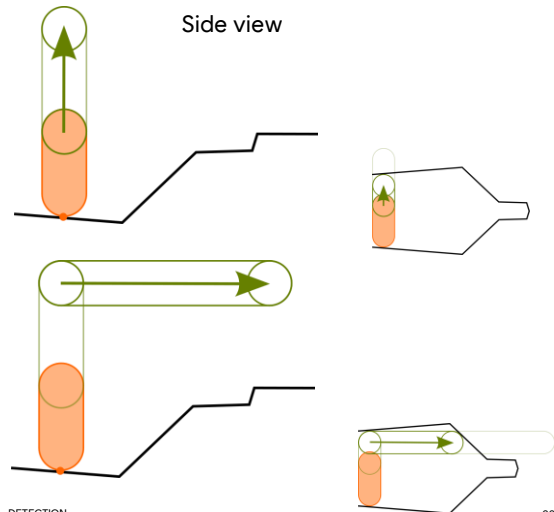
27

Now let's look at it in action

SEARCH PROCEDURE

- Up from a safe place

- Forward in input direction



So let's look at how we do this in practice.

We start by casting a sphere with the player radius up from a safe point within the capsule.

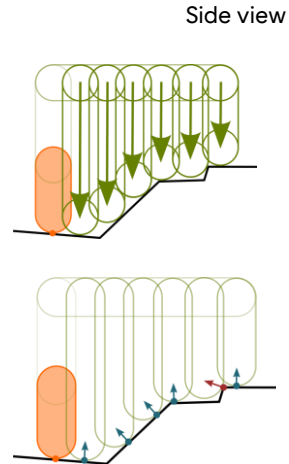
And this is a key point. All searches begin in a place that has been verified as physically safe. And we always make a continuous path from the capsule, which we know isn't intersecting anything. This is so that we can trust the results and to make sure that any point we find to move to is actually safe.

In the picture to the right you can see what happens if we hit something. We simply perform the next step from a lower height.

Once we've checked upwards we do a forward check in the input direction and again we continue from closer if we hit something

SEARCH PROCEDURE

- Down with overlapping samples
- Extract relevant data



PART 3 - DETECTION

29

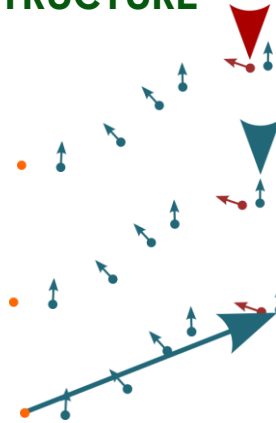
Once we have the spine of the profile gauge as it were, it's time for the pins.

We search down with overlapping spheres along the the whole spine. We actually do a little bit more, but we'll get back to that later.

After the down casts we extract the relevant data and act on it.

EVALUATION STRUCTURE

1. Is there an obstacle?
2. Can we stand beyond it?
3. Do we want to move there?



Ok. So we've got our data and now we want to find out if we want to traverse.

To do this we ask three questions. And these really are the core of the whole system.

The first question is: Is there an obstacle? Because if there is no obstacle, we don't need to do anything, we can just keep dragging our capsule

The next question is: Can we stand beyond it? Because if there is an obstacle but we can't get past it, there's nothing we can do.

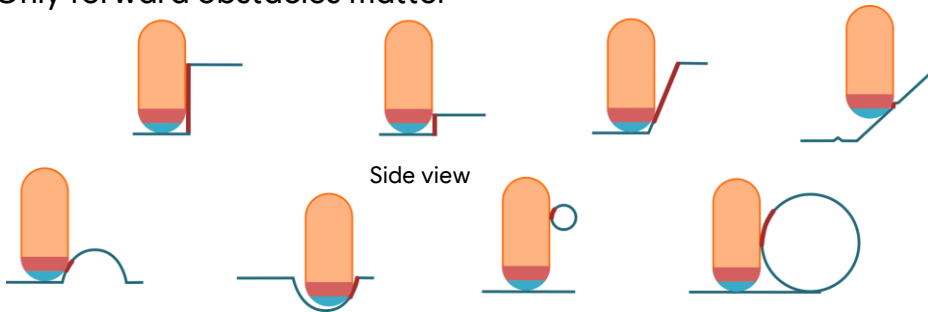
Finally we ask : Do we want to move there? Because if the time or the situation isn't good, we should wait.



So let's start with the first question: Is there an obstacle?

WHAT IS AN OBSTACLE?

- Generally: a part of the projected move path that the capsule would get stuck on
- Only forward obstacles matter



PART 4 - IS THERE AN OBSTACLE?

32

Before we can answer this question we need to define what an obstacle is. An obstacle for us is simply a part of the projected move path that the capsule would get stuck on. You can see a bunch of them in these pictures.

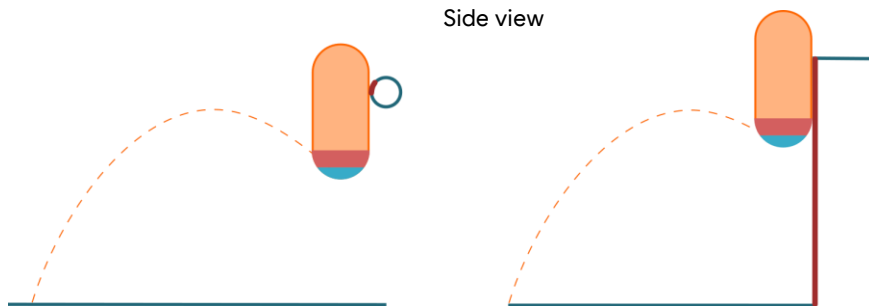
This can either be because of a hit in the unwalkable ring or because something hits higher up on the capsule.

Strictly speaking the unwalkable ring is actually an unwalkable hat covering everything above the walkable circle. But because our casts are vertical we only hit the bottom when we search.

And it's worth noting here that only obstacles in the forward direction matter. What we're moving away from or falling down from can't stop us.

WHAT IS AN OBSTACLE?

- Special case: In air



PART 4 - IS THERE AN OBSTACLE?

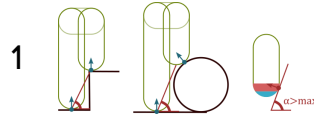
33

A bit of a special case here is when we're in air. The principle is the same. We're looking for things we can't get past. But rather than blocking dragging it blocks your trajectory.

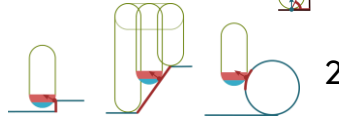
And sometimes, for big mantles to things like thin branches it doesn't even do that. But we can still use the same checks in air as on ground. So although it looks like a special case, it's all the same to the system.

HOW TO FIND AN OBSTACLE

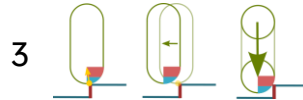
- 1: Hit height angle difference



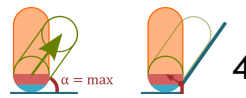
- 2: Unwalkable ring hit



- 3: Hit shift hit



- 4: Unwalkable capsule hit



PART 4 - IS THERE AN OBSTACLE?

34

So in order to find one of these obstacles we use four methods. They each have different strengths and weaknesses, but together they find essentially all obstacles eventually and the vast majority of objects quickly. Especially given iteration. We'll go through them in detail soon, but first I'll introduce them

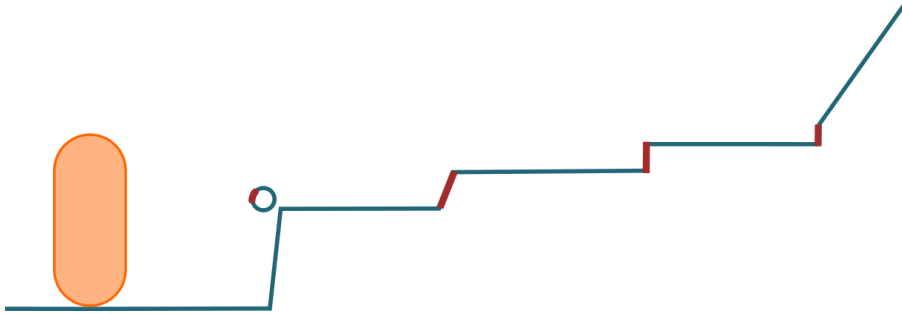
First we do a simple angle check. We just check if the slope between two samples is unwalkable.

Then we check for hits in the unwalkable ring. As I've mentioned this must be an obstacle

Then we move on to the special tricks. We do another unwalkable ring check, but this time armed with some targeting information

Finally we check for obstacles right in front of us as a fallback.

OUR OBSTACLE COURSE



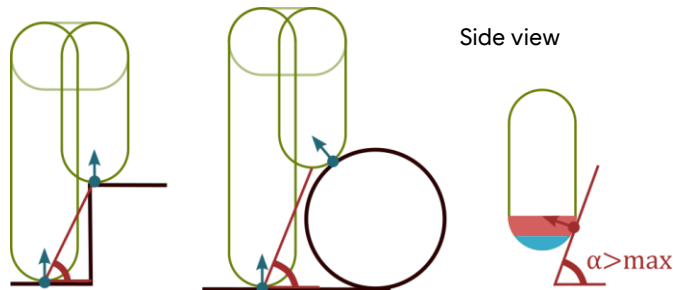
PART 4 - IS THERE AN OBSTACLE?

35

So, welcome to our obstacle course. We'll tackle these increasingly annoying obstacles using our four search methods.

1. HIT HEIGHT ANGLE DIFFERENCE

If the slope between two sphere bottoms is **unwalkable**, there is no way we can move past. There is an obstacle.



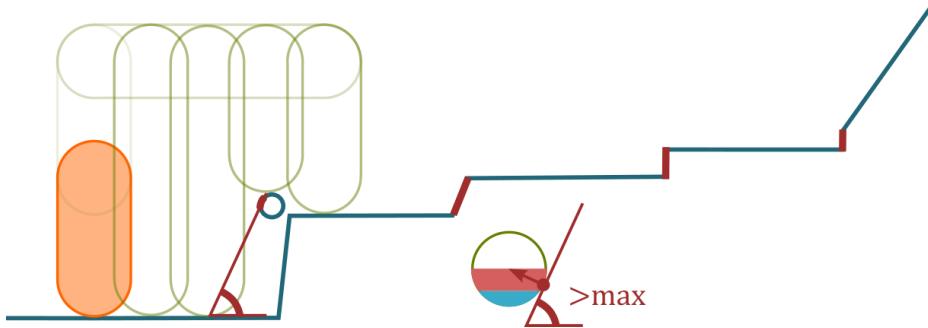
PART 4 - IS THERE AN OBSTACLE?

36

First out is hit height angle difference and it's pretty simple. We just look at two consecutive sphere bottoms and see if the slope between them is above the max angle. If it is, there must be an obstacle, because there is no way we could move from the first to the second.

1. HIT HEIGHT ANGLE DIFFERENCE

- **Strength:** Tall obstacles. Thin obstacles
 - Handles almost all mantles



PART 4 - IS THERE AN OBSTACLE?

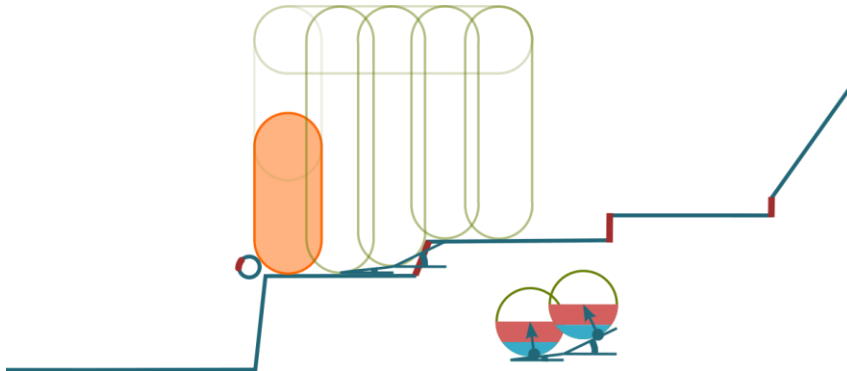
37

As you can see it found our little cylinder quite easily. This check is great at catching high obstacles, and it takes care of some very thin obstacles that the other methods can have some problems with.

In fact this handles almost all of the cases where we mantle and many of the stepups too. So at this point we've already caught a big part of the obstacles.

1. HIT HEIGHT ANGLE DIFFERENCE

- **Weakness:** Low obstacles



PART 4 - IS THERE AN OBSTACLE?

38

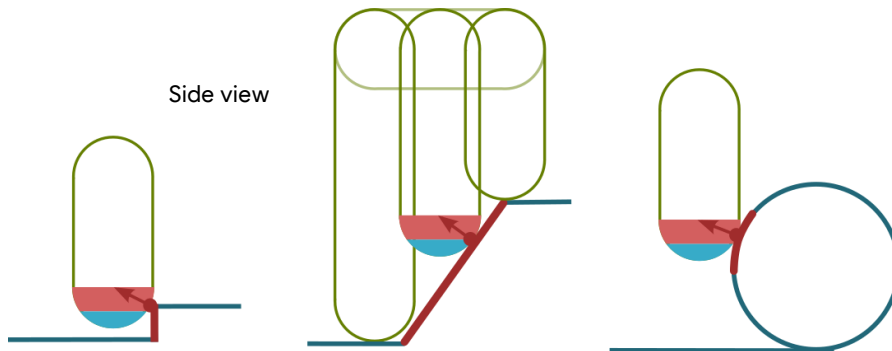
Angle difference has a major weakness though. As I mentioned we use a capsule partly because it smooths out motion. Unfortunately it also smooths out detection.

That's fine if it's a big obstacle, but if it's a small one it can disappear like with this next obstacle. As you can see, there is an obstacle, but because the capsule bottom dips below it the steep angle gets split into two shallow angles.

If you look closely at the cast that hits the obstacle though you can probably figure out comes next.

2.UNWALKABLE RING HIT

- A forward hit in the **unwalkable ring** means there is an obstacle



PART 4 - IS THERE AN OBSTACLE?

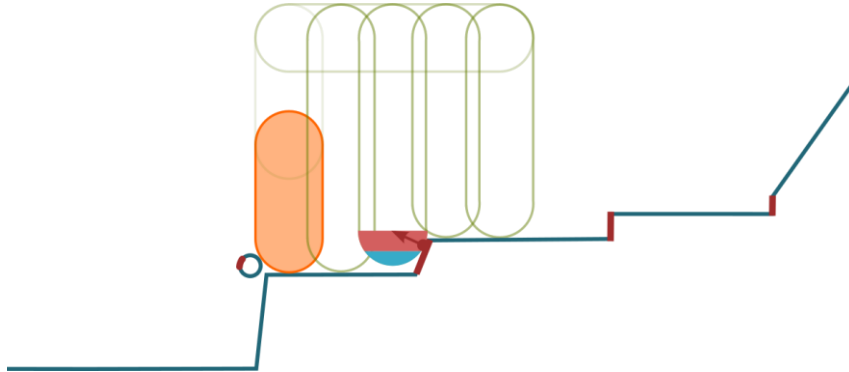
39

The unwalkable ring hit!

As we've already concluded a forward hit in the unwalkable ring means we have an obstacle. Because all normals in this area represent unwalkable slopes.

2.UNWALKABLE RING HIT

- **Strength:** Can find any obstacle from the right position



PART 4 - IS THERE AN OBSTACLE?

40

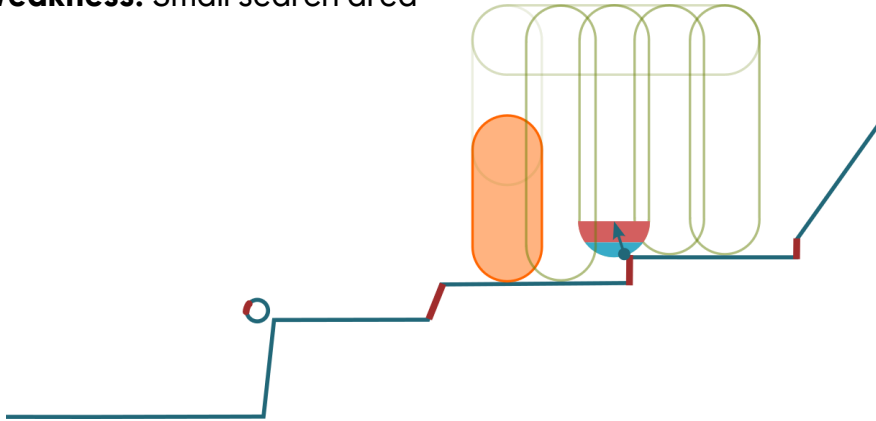
As you can see this found our obstacle from the last slide with the same sphere cast.

A great thing about this method is that as long as the cast is correctly positioned it can find any obstacle in our path. Or put another way, if we put the right part of unwalkable ring over the right part of the obstacle we'll find it.

So that's great right? This seem to solve the problem!

2.UNWALKABLE RING HIT

- **Weakness:** Small search area



PART 4 - IS THERE AN OBSTACLE?

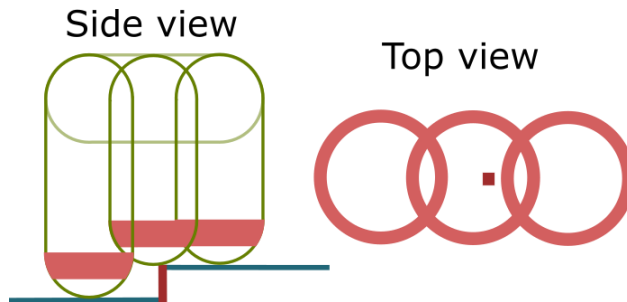
41

Well, not quite. The problem is that while it finds an obstacle if it's in the right position, it often isn't. Like here.

The only hit we get is in the walkable circle

2.UNWALKABLE RING HIT

- **Weakness:** Small search area
 - The **unwalkable ring** is small from above



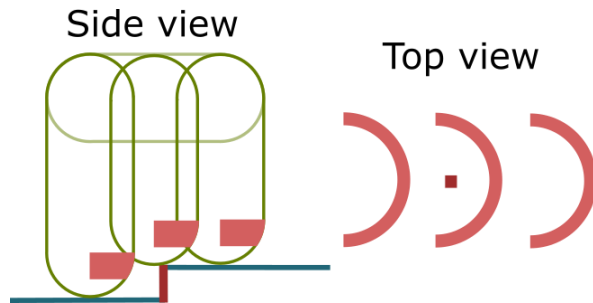
PART 4 - IS THERE AN OBSTACLE?

42

And this problem is actually worse than it looks. To begin with this ring that looks so large from the side is actually pretty thin when seen from above, because the steep angles become compressed by the projection.

2.UNWALKABLE RING HIT

- **Weakness:** Small search area
 - We only care about the front



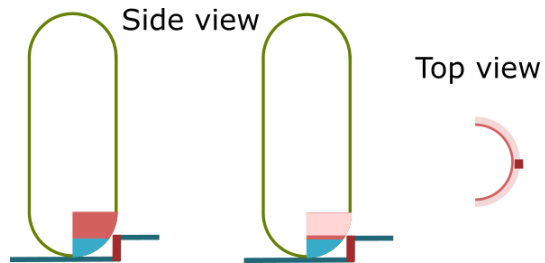
PART 4 - IS THERE AN OBSTACLE?

43

And we only use the front, so there goes half the search area.

2.UNWALKABLE RING HIT

- **Weakness:** Small search area
 - Small obstacles don't reach the upmost part of the **unwalkable ring**, making that part useless to search with



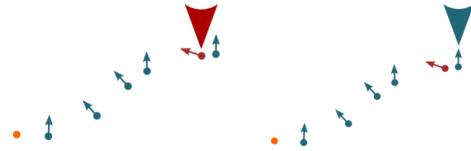
PART 4 - IS THERE AN OBSTACLE?

44

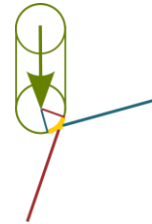
And it gets even trickier. If you look at this tiny little step it's high enough to block us but since it's so low it doesn't actually reach the top part of the unwalkable ring, so we can't get any hits there. So is there anything we can do?

3. HIT SHIFT HIT

- We're looking for an **unwalkable** slope followed by a **walkable** one



- For upwards slopes we know that the change from **steeper** to **shallower** slopes happens at convex corners



PART 4 - IS THERE AN OBSTACLE?

45

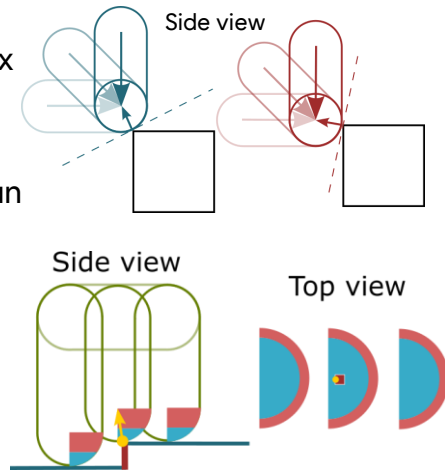
Luckily yes. But we'll have to think a bit.

First we consider our questions. "Is there an obstacle?" And "Can we stand beyond it?" In other words we're looking for something unwalkable followed by something walkable. Obstacle followed by a place to stand.

Now we know that this change can only happen when we move from something steeper to something shallower, and since we only care about upwards slopes, this means it must be a convex corner like the one you see pictured here. But how do we find it?

3. HIT SHIFT HIT

- Sphere casts are likely to hit convex corners
- If we know where a corner is, we can move the cast to it
- ...and we have a hit in the walkable circle. Let's guess it's a corner



PART 4 - IS THERE AN OBSTACLE?

46

Well, if you remember our earlier example in the sphere cast basics, we saw that sphere casts in different paths can hit the same corner, so we are pretty likely to hit corners to begin with.

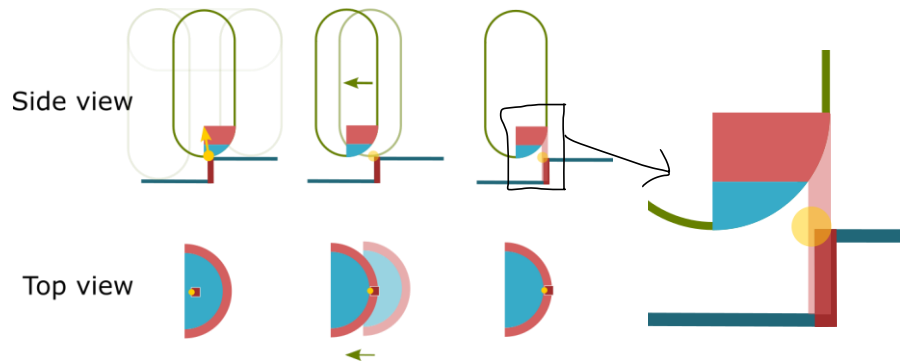
In fact we are more likely to hit corners (or edges in 3D) than any other specific points. Sphere casts love corners.

And if you look at this top example even more closely you'll see something else. They hit the same corner, but one is walkable and one is unwalkable. So what if we could move the first search so that it becomes the second? Walkable to unwalkable. Well, as it turns out we can.

If you remember our pesky little obstacle from the obstacle course, we did hit it, but only in the walkable circle, so it didn't register as an obstacle. Now let's just guess that it hit a convex corner and try again.

3. HIT SHIFT HIT

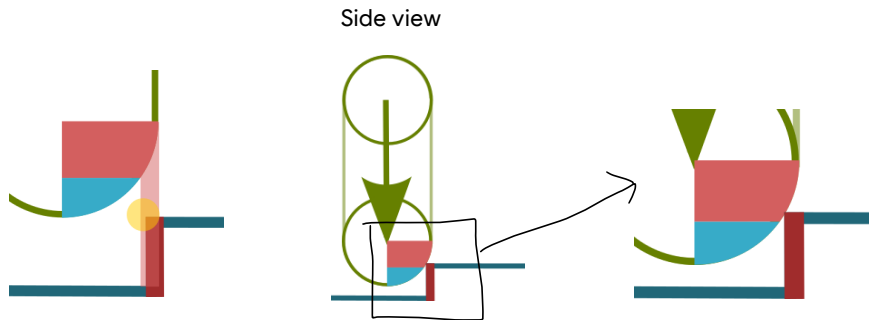
- Move a sphere back along the search direction to where the hit is inside the **unwalkable ring**



This brings us to the hit shift hit. All we do is to move a sphere back along the search direction until we're inside the unwalkable ring. You can see from above here as well how the yellow dot where we hit ends up under the unwalkable ring.

3. HIT SHIFT HIT

- Cast down. If the **unwalkable ring** hits, we have an obstacle

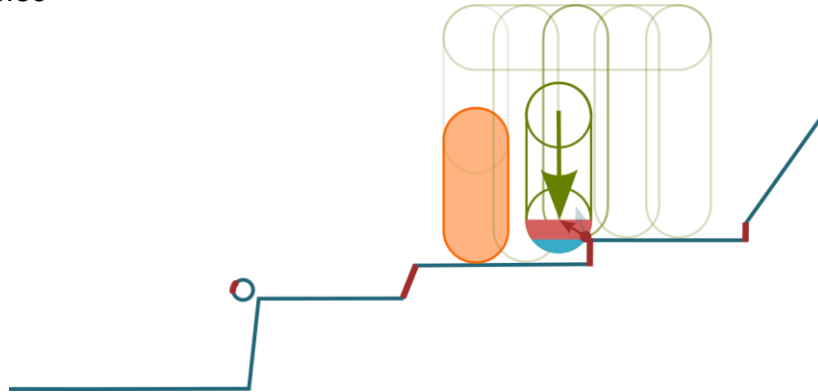


And then we cast down and see if it hits in the unwalkable ring. Since we know that a disproportionate amount of hit points are convex corners we are more likely to get an unwalkable hit this time. And in this case we did.

It's worth mentioning here that we don't know that it's a corner, and most of the time it's not, but we do this for every hit point anyway because it MIGHT be a corner.

3. HIT SHIFT HIT

Strength: Handles many of the remaining cases, including tiny obstacles



PART 4 - IS THERE AN OBSTACLE?

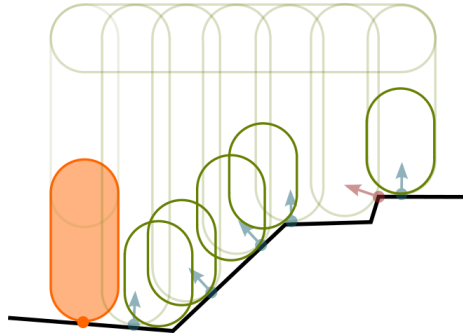
49

As you can see, just moving the cast to the hit point solved the problem with this obstacle.

This method caches a lot of our remaining cases including many small obstacles, and it makes a quite a difference in the quality of traversal.

3. HIT SHIFT HIT

Strength: Done together with other casts



PART 4 - IS THERE AN OBSTACLE?

50

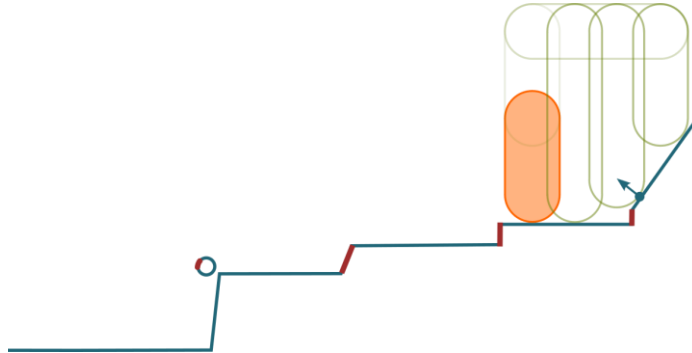
And we do this cast when we do the other downwards casts, so it's part of the profile gauge. And doing it right after and next to another cast makes it a bit cheaper.

So, problem solved?

3. HIT SHIFT HIT

Weakness: steep but walkable slopes

Convex corners are harder to find when the angle is obtuse



PART 4 - IS THERE AN OBSTACLE?

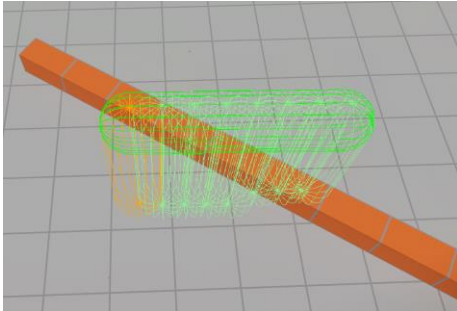
51

Of course not! Here for example, we hit the slope instead of the obstacle. That's because in general the more obtuse the angle of a convex corner, the harder it is to find. This one is very hard

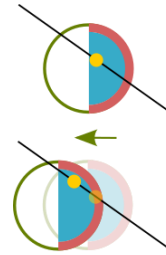
3. HIT SHIFT HIT

Weakness: Oblique long obstacles

These are easier to find for the basic check though



Top view



PART 4 - IS THERE AN OBSTACLE?

52

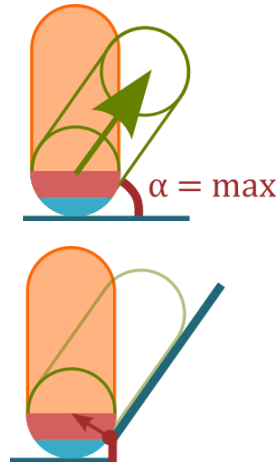
Another weakness is these long oblique obstacles. Because when we move the cast they shift the hit point we get and sometimes it ends up in the walkable circle again. Luckily though these are less of a problem to begin with since they are easier to find for the basic unwalkable ring check. Again, especially with iteration.

But there are more cases like this.

So what we need is a fallback.

4. UNWALKABLE CAPSULE HIT

- Represents the steepest path the capsule could travel
- Cast a sphere forward from the bottom at the max angle
- Any hit is an obstacle



PART 4 - IS THERE AN OBSTACLE?

53

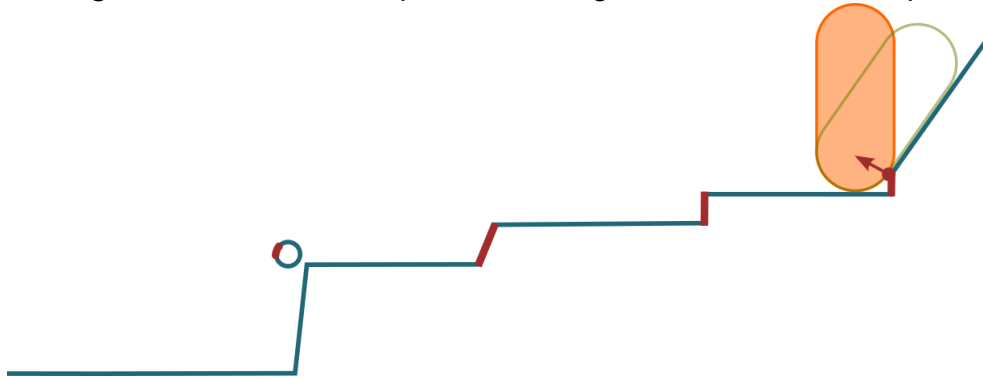
Enter the unwalkable capsule.

Here we simply imagine the steepest path we could possibly take to avoid any obstacles in our path. If something is in the way here, we know it's an obstacle because we can't avoid it by moving in a steeper angle.

So, we simply sphere cast forwards and upwards from the capsule bottom, from the feet, at the max angle and any hit is an obstacle.

4. UNWALKABLE CAPSULE HIT

Strengths: Finds essentially all remaining obstacles eventually



PART 4 - IS THERE AN OBSTACLE?

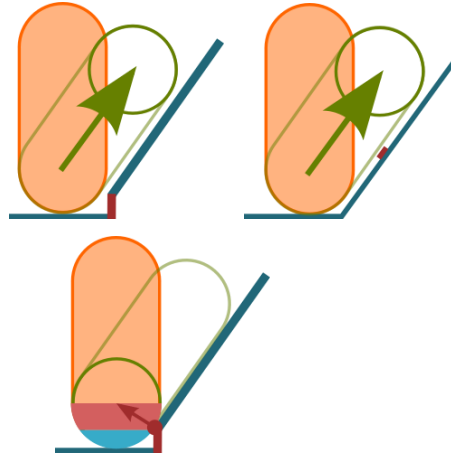
54

The advantage of this method is pretty dramatic. It finds pretty much any remaining obstacle eventually, including our final challenge in this obstacle course

4. UNWALKABLE CAPSULE HIT

Weaknesses:

- Only works right next to an obstacle
- Very late detection can cause stutter



PART 4 - IS THERE AN OBSTACLE?

55

The weakness though is equally dramatic.

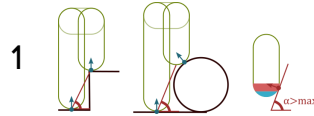
It only works when you're standing right next to an obstacle.

At this point you might have already hit the obstacle and got a little stutter in the movement.

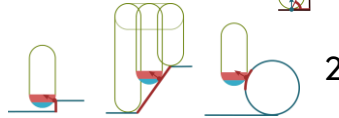
So it's not a great general solution. We'd much rather find obstacles with the other methods, and in the vast majority of cases we do, but it's still great to have something like this to make sure that failure isn't catastrophic.

RECAP: OBSTACLE SENSING

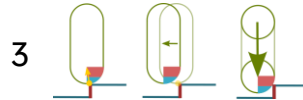
- 1: Hit height angle difference



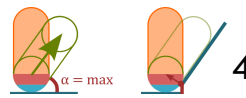
- 2: Unwalkable ring hit



- 3: Hit shift hit



- 4: Unwalkable capsule hit



PART 4 - IS THERE AN OBSTACLE?

56

Quick recap before we move on.

First we check if the angle between two consecutive samples is unwalkable.

Then we check if there are hits in the unwalkable ring.

Then we use the first hit to find a better place to try the unwalkable ring again.

And finally we test the steepest traversible path for obstacles.

That's it!

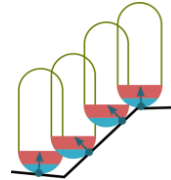


So, finally, can we stand beyond it?

CAN WE STAND BEYOND?

- We can stand at one of our searched positions if it has a hit in its walkable circle
- Walkable circles must overlap to find thin places to stand
- Any walkable hit beyond the first obstacle counts even if there are obstacles after

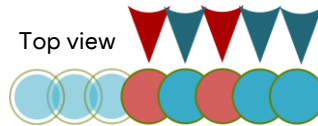
Side view



Top view



Top view



You'll be happy to know that this is a much simpler question. We can stand at any sampled position if the hit is in the walkable circle. It's that simple.

And as I mentioned before we use overlapping casts. More specifically we want the walkable circles to overlap or we might miss thin places to stand.

Once we have a walkable sample we simply check if there is any obstacle between that position and the character. If there are any obstacles beyond it doesn't matter. This happens in stairs for example and we want to be able to go to the first step.

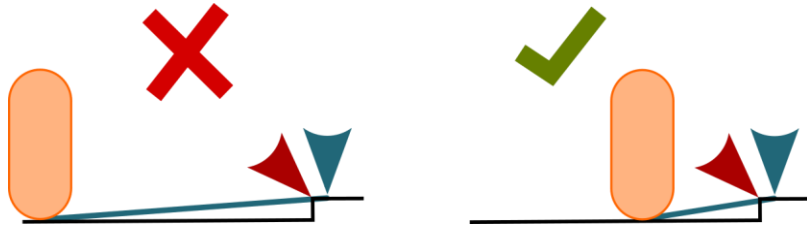
So that's it. We just check if a sample is walkable and if it's behind an obstacle. If the answer is yes, then it's a candidate for traversal.



And now the last of the evaluation questions before we commit to obstacle traversal.
Do we even want to move there?

WAIT FOR PATH STEEPNESS

Wait until the path is steep enough when far away



PART 6 - DO WE WANT TO MOVE THERE

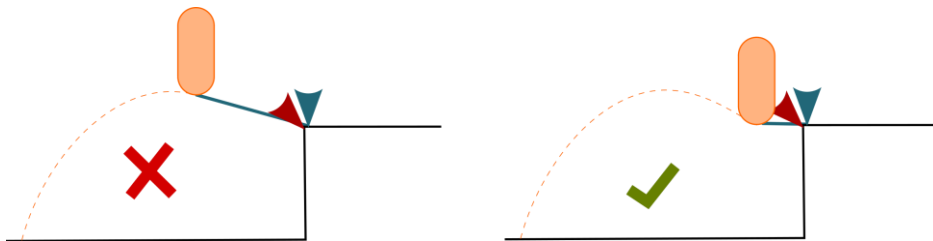
60

First we need to ask ourselves if maybe we should just wait a bit.

In this first example we have a candidate that would make for a very shallow stepup. Sort of like sliding onto a curb from three meters away. So when we're far away, we wait until it's steep enough.

LAND TO FLAT

Wait until we are level with the candidate



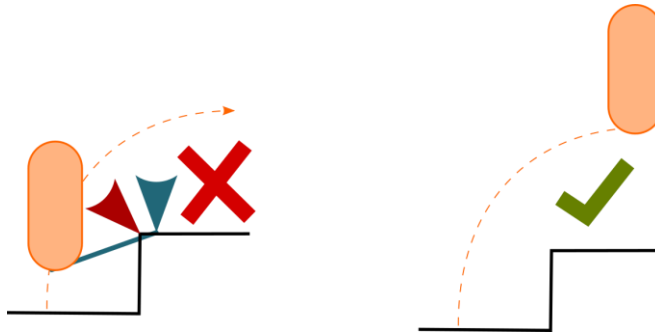
PART 6 - DO WE WANT TO MOVE THERE

61

In this example we'd rather not do an odd vampire glide to the candidate. It's better to wait until we're level with the candidate and make it look like a solid landing. In fact a big part of what feels like nailed jumps in our game are actually misses with flat stepups like this. Again, we want the player to feel competent.

LET THE PLAYER JUMP

Don't traverse to below jump peak



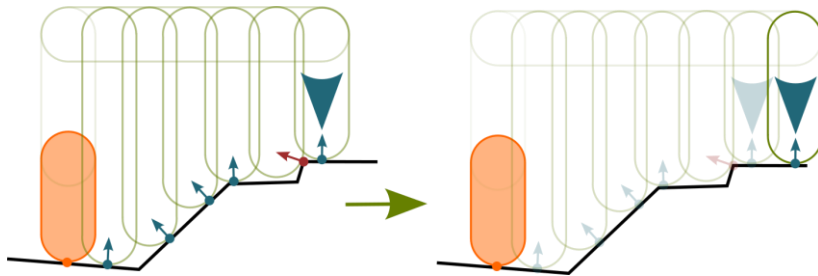
PART 6 - DO WE WANT TO MOVE THERE

62

The last one is the most important though. We really don't want the player to try to jump over something and then get stuck on it. So any candidates that are below the peak of a jump when the player is moving upwards are simply discarded.

THE JOKER – EXTRA PIN

If we have at least one valid candidate, look a little further



PART 6 – DO WE WANT TO MOVE THERE

63

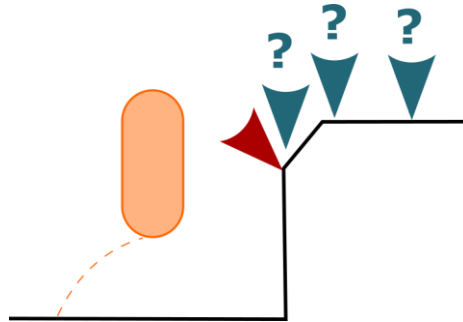
At this point I think it's worth mentioning a little special trick. If we have at least one valid candidate we look a little further by adding a pin to the end of our profile gauge.

It was there all along of course as part of the profile gauge casts, but we only look at it if we have a valid candidate.

This allows us to stay within metrics while also getting more candidates to choose from so we can make movement as smooth as possible.

CANDIDATE SCORING

Let's find the best candidate



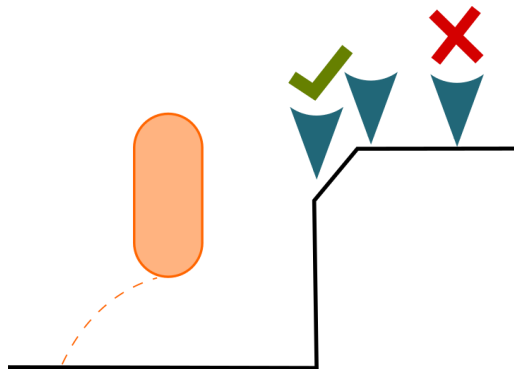
PART 6 - DO WE WANT TO MOVE THERE

64

So we've got our candidates that would be ok to move to. In this case three of them. But which one do we pick? Well, what we do is simply to score them all according to different factors and see which one scores the highest total. That's the one we move to. So let's look at the factors.

CANDIDATE SCORING

Short path – how close is it?



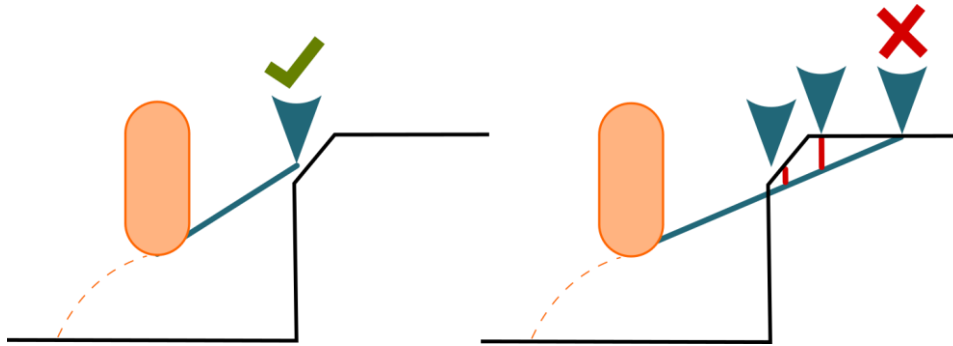
PART 6 – DO WE WANT TO MOVE THERE

65

First of all we like short paths because we don't want to take control away from the player. Short path, high score.

CANDIDATE SCORING

Low cutting – how little do we go through obstacles?



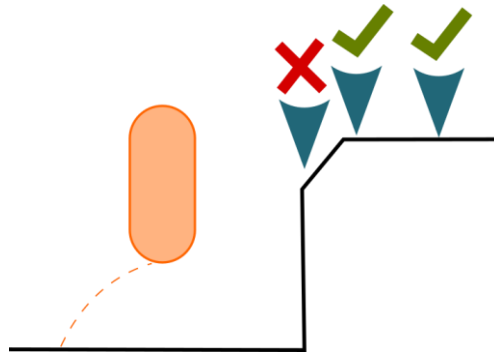
PART 6 – DO WE WANT TO MOVE THERE

66

We don't want to cut through geometry because it can look bad and makes it hard to abort. Low cutting, high score.

CANDIDATE SCORING

Flat arrival – how flat is the arrival slope?



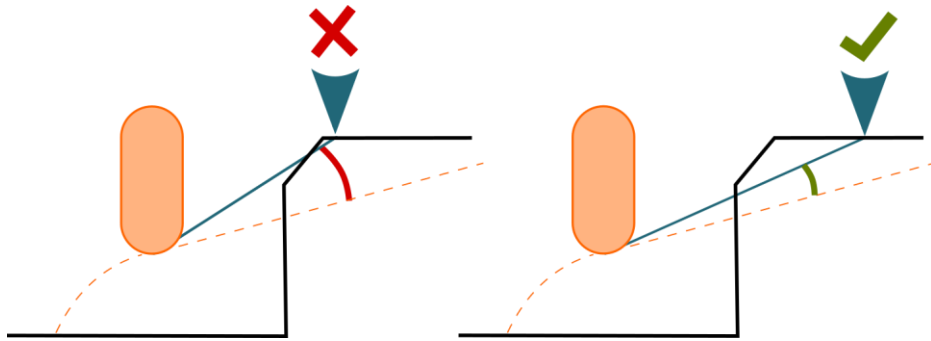
PART 6 – DO WE WANT TO MOVE THERE

67

We want to arrive somewhere that's as flat as possible, because this is more likely the top of an obstacle. This is especially relevant in our game where we have lots of cylindrical obstacles. Flat arrival, high score.

CANDIDATE SCORING

Incoming continuity– how aligned with current velocity



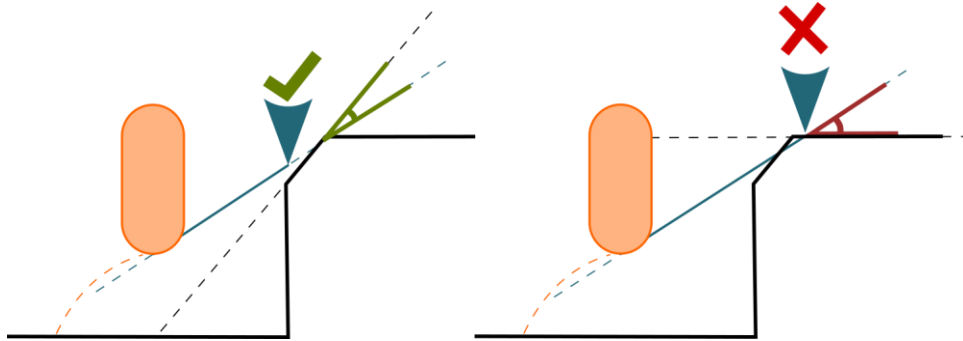
PART 6 – DO WE WANT TO MOVE THERE

68

We want the generated path to align with our incoming velocity because we want the start to feel smooth. So high incoming continuity, high score.

CANDIDATE SCORING

Outgoing continuity– how aligned with end slope



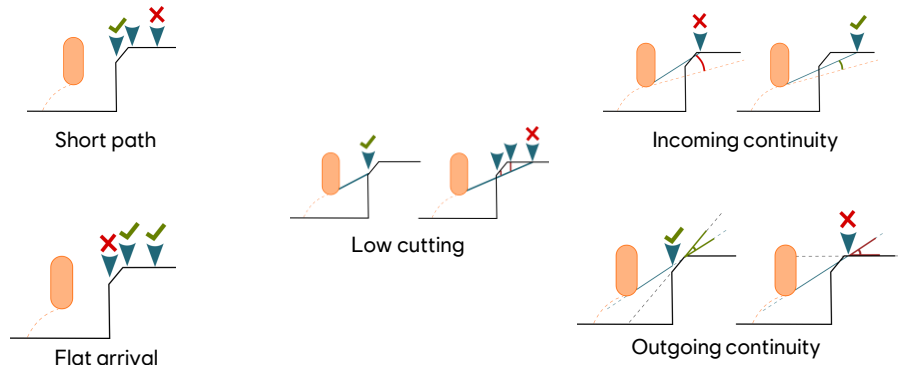
PART 6 – DO WE WANT TO MOVE THERE

69

And finally we want the path to align with the slope at the candidate position, because we want a smooth exit. High outgoing continuity, high score.

CANDIDATE SCORING

Now we weigh them together!



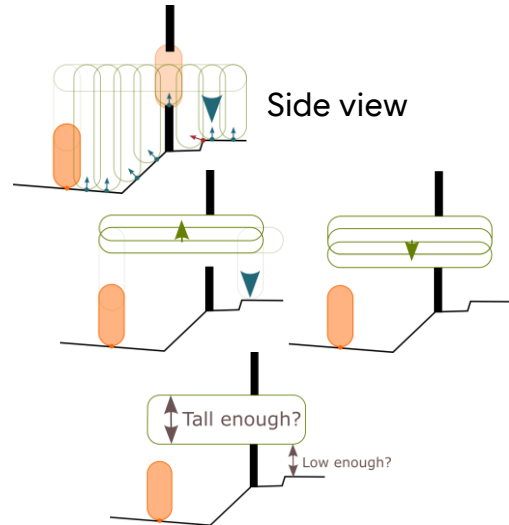
PART 6 - DO WE WANT TO MOVE THERE

70

Now, we just weigh them together. You may have noticed that different factors favour different candidates. And that's totally fine. We want to find the candidate that balances these factors best. That's why we weigh them together.

SAFETY CHECK – GETTING BACK

- The search may go through a small hole
- Cast the horizontal capsule up then down
- Is the resulting corridor tall enough and close enough to the candidate?



PART 6 – DO WE WANT TO MOVE THERE

71

Ok, so we have our candidate, but we need to make sure it's a safe one. More specifically we ask "If I move to this position, can I get back to where I started from"

Otherwise, we might end up in some hidden room because there's a small hole in the wall as in this picture. And by the way, that was an actual bug we had

Luckily this is pretty easy to check. If you remember the spine of our profile gauge is a horizontal cylinder with the player radius. So we know that the player fits sideways. All we need to do is to cast it up and then down to get a corridor. If the corridor is tall enough to fit the character and the bottom of it isn't too high above the candidate position we're good.

RECAP: EVALUATION STRUCTURE

1. Is there an obstacle?
2. Can we stand beyond it?
3. Do we want to move there?



PART 6 - DO WE WANT TO MOVE THERE

72

So, quick recap. We've just asked and answered the following questions.

Is there an obstacle?

Can we stand beyond it?

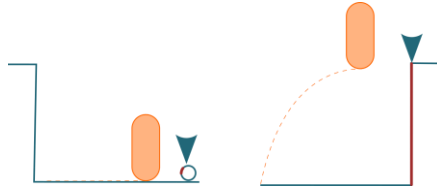
And Do we want to move there?



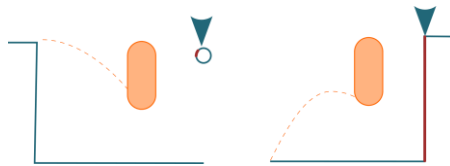
Time to move!

TYPE SELECTION

From ground or shallow:
stepup



From air and high:
mantle



PART 7 - MOVING

74

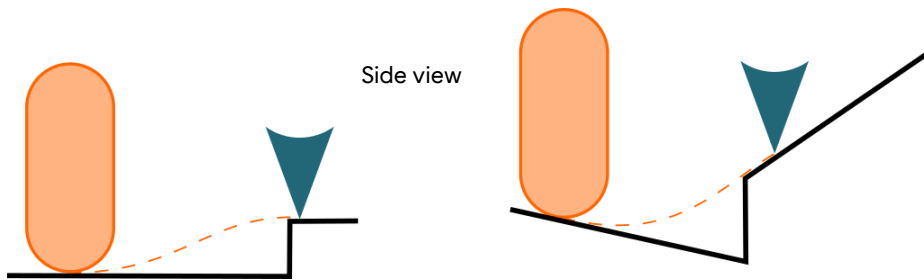
Once we've chosen where we want to go, we choose how we want to move there.

If we're on the ground or if we're in air but the traversal is shallow, we do a stepup. That's traversal using our legs.

If we're in air and the traversal is high, we do a mantle. That's using mainly our arms.

SPLINE GENERATION

General principle: Align with incoming velocity and outgoing slope



PART 7 - MOVING

75

Then, regardless of what type we choose, we generate a spline to move along.

The general principle is pretty simple. We essentially set the start tangent to the start velocity of the player and the end tangent to the arrival slope. As you can see here it's a bit like smoothing out the terrain.

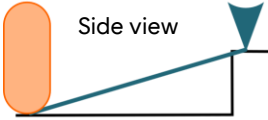
We vary this a bit for flavour, so if we do high stepups it's a bit more dramatic for example, and for mantles we modify the spline to suit the type of mantling, but in essence it's the same idea.

Once we have the spline, we just move along it.

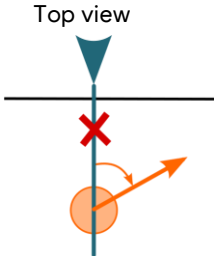
ABORT!

- If the input direction moves too far from the traversal direction: abort
- Must be outside world geometry

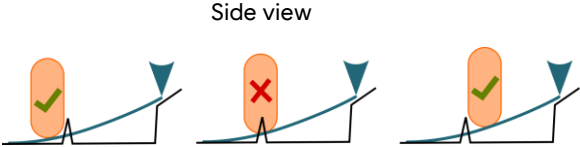
Side view



Top view



Side view



©

PART 7 - MOVING

76

So, movement is all very well, but what if I want to stop moving?

There can be several reasons for this. We could be thrown by an explosion or big animal for example. But far more common is that the player signals a change in intention. If the player stops giving input for example. Or, as in the top picture, if the input direction moves too far from the traversal direction, we try to abort.

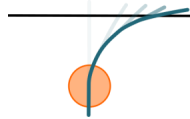
It's worth mentioning as well that we can't just abort anywhere. If we're currently clipping geometry with the capsule we wait until the coast is clear, and then we abort.



So that's all for the basic system! Before I wrap up I just want to go through some extensions that we made to it that I think add a lot.

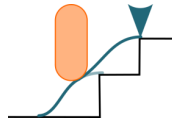
MOVEMENT EXTENSIONS

- Stepup Turning



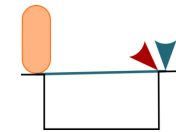
Top view

- Stair smoothing



Side view

- Gap traversal



Side view



PART 8 - EXTENSIONS

78

We'll start by looking at the extensions we made to make the movement smoother and more responsive

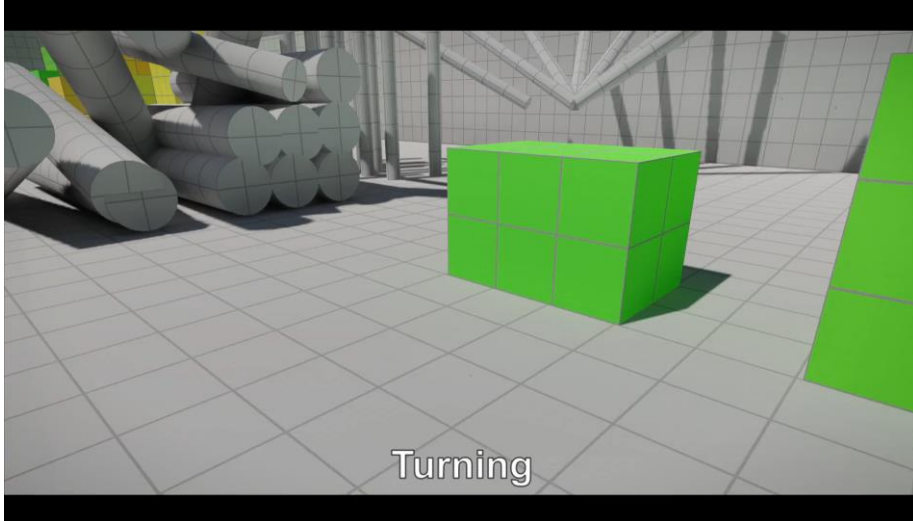
I'll go through how they work soon but first let's have a quick look.

We allow you to turn while you do a stepup to increase responsiveness and make the movement smoother

We smooth out stair like situations so make them feel less choppy

And we traverse over gaps so we can treat them as flat ground.

MOVEMENT EXTENSIONS IN ACTION



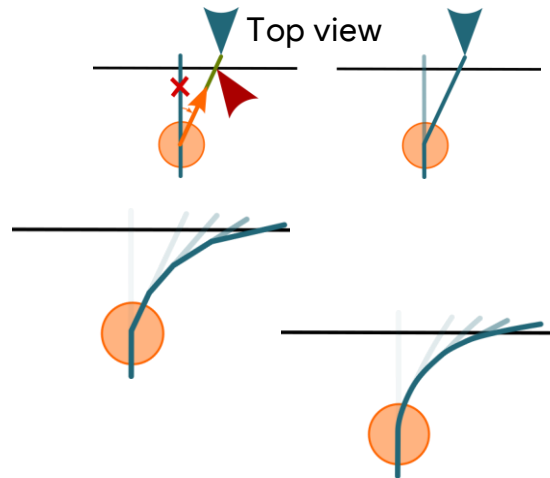
PART 8 - EXTENSIONS

79

So, before look at the details, let's see how they look in action!

STEPUP TURNING

- If we still have an obstacle and a place to stand, reduce abort angle
- Iteration creates a rough curve
- Spline continuity ensures smoothness



PART 8 - EXTENSIONS

80

So the first of the extensions is possibly my own favourite thing about the whole system. Turning during stepups.

This was actually a relatively late addition and came pretty naturally from how the system was built.

Even when we're engaged in obstacle traversal we still keep doing the same checks and the same analysis. So we always still know if there is an obstacle ahead and somewhere to stand beyond it.

This makes turning pretty trivial. We just check if we have a valid candidate for stepup ahead. If we do we simply reduce the abort angle to a few degrees. Now if the player turns, we can just abort and be sure that a new stepup will start immediately in the new angle.

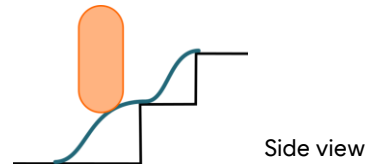
As you can see in the middle image this forms a rough curve as we abort, replan, abort, replan

And when you add the smoothing that the spline gives us, you get a pretty smooth

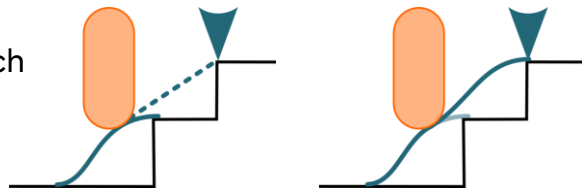
turn. In fact with small stepups, most of the time our players don't even realize it's happening, because they can move and turn like normal. So for all intents and purposes it's like moving on the ground.

STAIR SMOOTHING

- Stairs can get pretty choppy with full move, followed by immediate start



- If a significantly steeper stepup is available, we switch to that



PART 8 - EXTENSIONS

81

Stair smoothing uses a very similar principle.

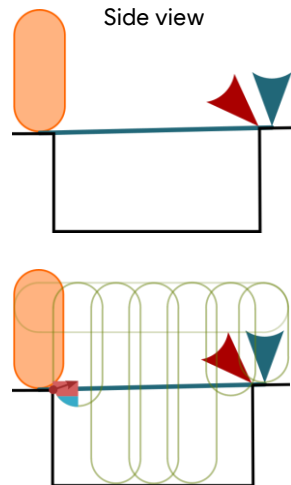
When we move up stair like structures the motion can get pretty choppy if we try to finish the move to each step before we start with the next.

So what we do is simply to check if there is a new stepup available that's steeper than what's left of the current one. If that's the case we just start with that without waiting for the current one to finish.

You can see how much smoother it gets in the lower image.

GAP TRAVERSAL

- Normally we skip stepups this shallow from this far away
- We ignore this rule when back obstacles are close
- Back obstacles are found like front obstacles but in reverse



PART 8 - EXTENSIONS

82

Here's a slightly more complex one. And I have to admit here that I lied a bit earlier.

I said that we only care about front obstacles. Well, that's not really true. We actually look for obstacles in the same way backwards. That doesn't mean we do a separate set of casts, just that we run the analysis both ways.

So why would we do that? Well, If you look at this top picture, that's a very shallow step up far away, so we would usually discard it, but as you can see we're about to fall into a pit, and we don't want that.

So all we do is to look if we have any backwards obstacles close to us. If there are, we just ignore the steepness rule. This allowed us to treat some pretty tricky geometry as essentially flat ground.

MARKUP – NO TRAVERSAL



PART 8 - EXTENSIONS

83

Markup time.

I wrote in the description of this talk that the system requires almost no markup. Actually it requires no markup, but it's still useful in some cases.

The first of our three types of markup is simply the ability to tag collision as not traversible. This will still make them register as obstacles but you can't stand there. This is used for things we don't want the player to get onto or simply to wall off sections and be sure that we can't get past.

MARKUP – ONLY EXPLICIT TRAVERSAL



PART 8 – EXTENSIONS

84

The next type is a bit more interesting. There are objects in the game that we don't want to automatically step up on or over, but we also don't want to have the player get stuck on them. Sometimes that's because they're interactable, but another example is holes like this, where we don't want the player to step over it, because it's part of the mission path, but we want them to be able to get back out again once they're down there.

What we do in this case is to only allow traversal to this object when you are in what we call a ballistic air state. That's things like jumping, bouncing or getting thrown by an explosion. Essentially it has in some way been signalled that we want to get onto stuff.

MARKUP – HUMAN INTERIOR POLITENESS VOLUME



PART 8 – EXTENSIONS

85

The final one's about the woes of being three meters tall. So in the game we're friends with these humans in the resistance. But they're tiny, and so are their things. So this kitchen table looks a lot like a rock in the forest to our system and it's really tempting to step onto it. Which feels pretty rude.

So we made a volume for places like this where we tell traversal to calm down a bit. So, the search height is lower and the distance is shorter. We also lowered the speed a bit for good measure. This made us feel much more polite.



Almost done, just three slides left!

PERFORMANCE

- Might not be for an MMO, but not as bad as it sounds
- Casts against player collision only
- Tightly packed casts give cache hits
- The data from sensing is reused



PART 9 - WRAPPING UP

87

At this point I can hear some of you thinking "But what about the performance with all those casts?"

Well, it's probably not the top candidate for your next MMO but it's not as bad as you might think. On a getting-on-a-bit PC, even in a dense jungle environment the whole thing doesn't get much above the low hundreds of microseconds. And in usually it's much less. And that is without any major optimization effort beyond the basics.

The reason for this is first of all that we only cast against player collision as opposed to more complex projectile collision.

We also do casts tightly packed in time and space which means that if we do them sequentially we get cache hits. And we actually had a significant performance boost by doing them on one thread rather than many.

Another interesting performance aspect is sensor data reuse. Once we have this wonderful data, we can use it for other things, and we do.

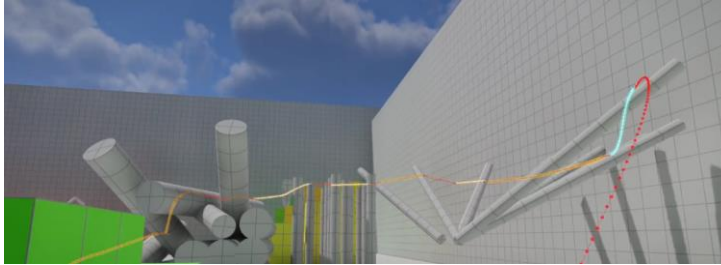
Our side jump for example is really shallow. Like a dodge. And if we're standing next

to a steep slope, we don't want to jump into it. So we use this data to analyze the slope and adjust the jump to it.

We also use it for calculating something we call a convexness value that we use to smooth out the movement.

LESSONS LEARNED

- Active simplification pays
- Debug drawing is a very good thing



- Generous help makes it hard to block paths



PART 9 - WRAPPING UP

88

Now let's briefly look at some lessons learned.

The first one is that active simplification pays. In fact all our extensions; stepup turning, stair smoothing and gap traversal were essentially freebies. They took just a few days to make in total. But that's just because we had all the answers we needed ready from the system.

The next point is pretty obvious, but it's something that I at least learn a little bit more with every year. Debug drawing helps a lot. As an example we added the debug tracks that you might have noticed in the extensions video a bit into the project and they made a huge difference. We have them for a whole bunch of different positions, like the player position, the camera, the aim point and so on, and we added a lot of state information to it. This meant that if something felt off you could just turn around and find it. This helped us a lot in quickly finding and fixing unsmooth movement.

The third one is about being careful what you wish for. We did all we could to make a system that could find a way past any obstacle. And, well it did. A little bit too well. Now this might sound like a humble brag, but this turned into a real headache for our

level designers. Because if you want to wall something off in a jungle it makes sense to add something like a rock wall. But a rock wall usually has some tiny ledges and our system found them and used them to climb up, which allowed players to go into all the forbidden areas and do all the forbidden things. So in the end we solved it by adding some no traversal flags, some slippery slopes and some invisible collision, but I wish we'd thought of this earlier and come up with better solutions.

FUTURE WORK

- Optimization
- More sensor reuse
- Turning without abort



Final slide. Future work. An this is just some things I'd like to look at more.

An important part of course is exploring optimization options. This was never an issue for us, but I think a lot could be gained here. We can look at things like explicit geometry caching, batch casting, cast distribution, and more specific things like checking in the sphere cast if it was an edge hit and not just guessing for each one.

Related to the first point is to look more at what else we can use this information for. Because once we have this information we might be able to use it for things like foot planting or anticipatory animation for example. Because we know where the gaps and the obstacles are and how the slope changes.

Finally the abort and replan principle. This trick of just aborting and replanning all the time was a really convenient trick and it allowed us to make things very quickly, but I feel that modifying the spline instead could probably make it feel even nicer.

